

UNIVASSOURAS
ENGENHARIA DE SOFTWARE

LEANDRO LIMA CARDOSO (202323366)

DESENVOLVIMENTO DE UMA *GAME ENGINE* 2D EM FORMATO DE BIBLIOTECA
EM C++
PRÁTICAS EXTENSIONISTAS INTEGRADORAS VI

SAQUAREMA
2026

LEANDRO LIMA CARDOSO (202323366)

**DESENVOLVIMENTO DE UMA *GAME ENGINE* 2D EM FORMATO DE BIBLIOTECA
EM C++**

PRÁTICAS EXTENSIONISTAS INTEGRADORAS VI

Trabalho apresentado como requisito parcial para obtenção de nota da disciplina de **Práticas Extensionistas Integradoras VI** do curso de Engenharia de Software da Univassouras (campus universitário de Saquarema).

Prof. Dr. Altemar Sales

SAQUAREMA

2026

RESUMO

Este trabalho apresenta o desenvolvimento de uma *game engine* 2D implementada em C++, destinada à criação de jogos baseados em *tiles* com visão *top-down*. O projeto tem como objetivo estruturar uma arquitetura modular e performática, minimizando a dependência de bibliotecas externas e proporcionando maior controle sobre gerenciamento de memória, renderização gráfica e organização de recursos. A pesquisa caracteriza-se como aplicada e exploratória, fundamentada em revisão bibliográfica sobre engenharia de software, padrões de projeto e arquitetura de sistemas. A metodologia adotada envolve modelagem arquitetural, implementação incremental dos módulos da engine e validação por meio da construção de um protótipo funcional de jogo. São abordados aspectos como sistema de renderização 2D, gerenciamento de entidades, controle de colisões e administração de *assets*. Como resultado esperado, busca-se obter uma *engine* leve, especializada e de fácil utilização para jogos estruturados em mapas compostos por *tiles*, contribuindo para o aprofundamento acadêmico em arquitetura de software e otimização de desempenho em aplicações gráficas.

Palavras-chave: *game engine*; C++; arquitetura de software; jogos 2D; desempenho.

SUMÁRIO

1 INTRODUÇÃO.....	5
1.1 Contexto.....	6
1.2 Problema.....	6
1.3 Justificativa.....	7
1.4 Objetivos.....	7
1.4.1 Objetivo Geral.....	7
1.4.2 Objetivos Específicos.....	8
2 REFERENCIAL TEÓRICO.....	8
2.1 Engenharia de Software e Arquitetura de Sistemas.....	8
2.2 Padrões de Projeto e Modularidade.....	9
2.3 <i>Game Engines</i> : Conceito e Estrutura.....	9
2.4 Renderização 2D e Sistemas Baseados em <i>Tiles</i>	10
2.5 Linguagem C++ e Desempenho.....	10
2.6 Gerenciamento de Recursos e Entidades.....	11
3 METODOLOGIA.....	11
3.1 Procedimentos Técnicos.....	12
3.2 Modelagem Arquitetural.....	12
3.3 Implementação Incremental.....	13
3.4 Minimização de Dependências Externas.....	14
3.5 Validação do Sistema.....	14
3.6 Limitações Metodológicas.....	15
4 PROJECT CANVAS.....	15
4.1 Descrição do Project Canvas.....	15
4.1.1 Problema.....	16

4.1.2 Solução Proposta.....	16
4.1.3 Produto Final.....	16
4.1.4 Público-Alvo.....	17
4.1.5 Objetivos do Projeto.....	17
4.1.6 Entregas Principais.....	17
4.1.7 Premissas.....	17
4.1.8 Restrições.....	18
4.1.9 Riscos Iniciais.....	18
5 GERENCIAMENTO DO ESCOPO.....	18
5.1 Objetivos SMART.....	19
5.2 Produto e Entregas.....	20
5.3 Matriz de Rastreabilidade dos Requisitos.....	21
5.4 Premissas.....	21
5.5 Restrições.....	21
5.6 Estrutura Analítica do Projeto (EAP).....	22
5.7 Principais Atividades e Estratégia do Projeto.....	22
6 GERENCIAMENTO DO TEMPO.....	23
6.1 Definição das Atividades.....	23
6.2 Sequenciamento das Atividades.....	25
6.3 Estimativa de Duração.....	26
6.4 Marcos do Projeto.....	26
6.5 Cronograma (Modelo para Gráfico de Gantt).....	27
6.6 Controle do Cronograma.....	27
7 EQUIPE.....	28
7.1 Estrutura Organizacional do Projeto.....	28

7.2 Responsabilidades.....	29
7.3 Matriz de Responsabilidades (RACI Simplificada).....	29
7.4 Competências Necessárias.....	30
7.5 Estratégia de Desenvolvimento Individual.....	30
7.6 Plano de Comunicação Interna.....	31
8 GERENCIAMENTO DAS COMUNICAÇÕES.....	31
8.1 Objetivos da Comunicação.....	31
8.2 Partes Interessadas na Comunicação.....	32
8.3 Canais de Comunicação.....	32
8.4 Frequência das Comunicações.....	32
8.5 Registro e Armazenamento de Informações.....	33
8.6 Tipos de Informação Comunicada.....	33
8.7 Fluxo de Comunicação.....	33
9 GERENCIAMENTO DAS PARTES INTERESSADAS.....	34
9.1 Identificação das Partes Interessadas.....	34
9.2 Análise das Partes Interessadas.....	34
9.3 Matriz Interesse x Influência.....	35
9.4 Estratégias de Engajamento.....	35
9.5 Monitoramento das Partes Interessadas.....	36
10 GERENCIAMENTO DOS RISCOS.....	37
10.1 Identificação dos Riscos.....	37
10.2 Análise Qualitativa dos Riscos.....	37
10.3 Estratégias de Resposta aos Riscos.....	38
10.4 Plano de Contingência.....	39
10.5 Monitoramento dos Riscos.....	39

11 MONITORAMENTO E CONTROLE.....	40
11.1 Controle do Escopo.....	40
11.2 Controle do Cronograma.....	41
11.3 Controle da Qualidade.....	41
11.4 Indicadores de Desempenho.....	42
11.5 Relatórios de Acompanhamento.....	42
11.6 Processo de Ajustes.....	42
12 CONTROLE INTEGRADO DE MUDANÇAS.....	43
12.1 Objetivo do Controle de Mudanças.....	43
12.2 Tipos de Mudanças Possíveis.....	44
12.3 Processo de Solicitação de Mudança.....	44
12.4 Critérios para Aprovação de Mudanças.....	44
12.5 Registro das Mudanças.....	45
12.6 Autoridade de Aprovação.....	46
12.7 Controle de Versões.....	46
13 LIÇÕES APRENDIDAS.....	47
14 REQUISITOS ESPECÍFICOS.....	49
14.1 Requisitos Funcionais.....	49
14.2 Requisitos Não Funcionais.....	51
15 VISÃO GERAL DA ARQUITETURA.....	53
16 ESTILO ARQUITETURAL.....	54
17 COMPONENTES DO SISTEMA.....	56
17.1 <i>Core</i>	56
17.2 <i>Renderer</i>	57
17.3 <i>Entity System</i>	57

17.4 Collision System.....	58
17.5 Input System.....	58
17.6 Resource Manager.....	58
17.7 Scene Manager.....	59
18 DESCRIÇÃO DOS MÓDULOS.....	59
18.1 Módulo Core.....	59
18.2 Módulo de Renderização.....	60
18.3 Módulo de Entidades.....	61
18.4 Módulo de Colisão.....	61
18.5 Módulo de Entrada.....	62
18.6 Módulo de Gerenciamento de Recursos.....	62
18.7 Módulo de Gerenciamento de Cenas.....	63
19 FLUXO DE EXECUÇÃO DA ENGINE.....	63
19.1 Inicialização do Sistema.....	64
19.2 Processamento de Entrada.....	64
19.3 Atualização da Lógica do Jogo.....	65
19.4 Detecção de Colisões.....	65
19.5 Renderização da Cena.....	66
19.6 Encerramento da Aplicação.....	66
20 Diagramas.....	66
20.1 Caso de Uso.....	66
20.2 Diagrama de Sequencia.....	68
20.3 Classe e Relacionamento.....	68
REFERÊNCIAS.....	69

1 INTRODUÇÃO

A indústria de jogos digitais consolidou-se como um dos segmentos mais relevantes da indústria de software contemporânea, impulsionando avanços significativos nas áreas de computação gráfica, arquitetura de sistemas, processamento paralelo e otimização de desempenho. O desenvolvimento de jogos digitais demanda integração entre múltiplas disciplinas, incluindo engenharia de software, modelagem matemática, design de interação e programação orientada a objetos. Nesse contexto, as *game engines* assumem papel central, pois fornecem infraestrutura reutilizável responsável pela abstração de funcionalidades complexas como renderização gráfica, gerenciamento de memória, controle de entrada e organização de entidades.

Segundo Sommerville (2019), a definição adequada da arquitetura de software é um dos fatores mais relevantes para garantir qualidade, manutenibilidade e desempenho em sistemas complexos. No desenvolvimento de motores gráficos, decisões arquiteturais impactam diretamente na escalabilidade e eficiência do sistema. De forma complementar, Gamma et al. (2000) destacam que a aplicação adequada de padrões de projeto contribui para modularidade, reuso e organização estrutural do código, aspectos fundamentais em projetos que envolvem múltiplos subsistemas interdependentes.

Atualmente, motores amplamente utilizados no mercado, como Unity e Unreal Engine, oferecem soluções completas para desenvolvimento de jogos, incluindo sistemas gráficos avançados, física, animação e ferramentas visuais integradas. Entretanto, tais plataformas apresentam elevado nível de abstração, grande dependência de bibliotecas externas e arquitetura voltada para múltiplos tipos de jogos, o que pode resultar em complexidade excessiva para projetos específicos de menor escopo, especialmente no contexto educacional.

Nesse cenário, observa-se a necessidade de compreender de forma aprofundada os mecanismos internos que compõem uma *game engine*, particularmente no que se refere

ao gerenciamento de recursos, renderização 2D e estruturação modular. A linguagem C++ apresenta-se como alternativa adequada para esse propósito, devido ao seu controle refinado sobre memória e recursos computacionais, além de seu amplo uso em aplicações de alto desempenho (STROUSTRUP, 2013).

1.1 Contexto

Jogos 2D baseados em mapas compostos por *tiles* com visão *top-down* permanecem amplamente utilizados tanto em produções independentes quanto em ambientes acadêmicos. Esse modelo estrutural organiza o cenário em unidades gráficas discretas, permitindo controle eficiente de colisão, movimentação e renderização. A simplicidade conceitual desse formato torna-o apropriado para experimentação arquitetural e implementação incremental de funcionalidades típicas de motores gráficos.

Além disso, o desenvolvimento de uma *game engine* especializada em jogos 2D possibilita maior foco na otimização de subsistemas essenciais, como gerenciamento de entidades, controle de eventos, carregamento de recursos e organização de camadas de renderização, evitando a complexidade inerente a motores generalistas.

1.2 Problema

Apesar da disponibilidade de motores consolidados no mercado, observa-se que soluções generalistas frequentemente incorporam múltiplas camadas de abstração e dependências externas que não são estritamente necessárias para jogos 2D baseados em *tiles*. Essa característica pode dificultar o entendimento aprofundado da arquitetura interna do motor e limitar a experimentação acadêmica relacionada a desempenho e organização estrutural.

Dessa forma, formula-se o seguinte problema de pesquisa:

Como desenvolver uma *game engine* 2D em formato de biblioteca, utilizando C++, que seja modular, performática e com mínima dependência de bibliotecas externas, facilitando a criação de jogos baseados em *tiles* com visão *top-down*?

1.3 Justificativa

Do ponto de vista acadêmico, o desenvolvimento de uma *game engine* própria possibilita aplicação prática de conceitos fundamentais da engenharia de software, tais como definição arquitetural, modularização, encapsulamento, reuso de código e documentação técnica. Conforme destaca Sommerville (2019), sistemas bem estruturados arquiteturalmente tendem a apresentar maior facilidade de manutenção e evolução.

Sob a perspectiva técnica, a implementação direta de componentes essenciais como renderização 2D, gerenciamento de memória e organização de entidades, permite compreender detalhadamente o impacto das decisões arquiteturais no desempenho do sistema. Essa abordagem favorece o desenvolvimento de competências relacionadas à otimização de recursos computacionais, aspecto particularmente relevante em aplicações gráficas.

Adicionalmente, ao estruturar a *game engine* em formato de biblioteca, amplia-se sua possibilidade de reutilização em diferentes projetos, promovendo separação clara entre motor e aplicação final, característica alinhada às boas práticas de projeto de software (GAMMA et al., 2000).

1.4 Objetivos

1.4.1 Objetivo Geral

Desenvolver uma *game engine* 2D em formato de biblioteca, utilizando a linguagem C++, com foco em desempenho, modularidade e baixa dependência de bibliotecas externas, destinada à criação de jogos baseados em *tiles* com visão *top-down*.

1.4.2 Objetivos Específicos

- Projetar a arquitetura modular da *engine*, definindo camadas e responsabilidades.
- Implementar sistema de renderização 2D otimizado para mapas baseados em *tiles*.
- Desenvolver mecanismo de gerenciamento de recursos gráficos.
- Estruturar sistema de entidades e controle de colisão.
- Minimizar o uso de dependências externas, priorizando implementações próprias.
- Validar a *engine* por meio do desenvolvimento de um protótipo funcional.
- Documentar decisões arquiteturais e resultados obtidos.

2 REFERENCIAL TEÓRICO

2.1 Engenharia de Software e Arquitetura de Sistemas

A engenharia de software estabelece princípios e práticas voltados à construção de sistemas confiáveis, eficientes e manuteníveis. De acordo com Sommerville (2019), a arquitetura de software representa a estrutura fundamental de um sistema, definindo seus componentes, suas interações e as decisões de alto nível que impactam atributos de qualidade como desempenho, escalabilidade e manutenibilidade.

Em sistemas que envolvem processamento gráfico e manipulação intensiva de recursos, as decisões arquiteturais assumem papel ainda mais crítico. Pressman e Maxim (2020) destacam que a organização modular do software contribui para redução da

complexidade e facilita a evolução incremental do sistema, especialmente em projetos com múltiplos subsistemas interdependentes.

No contexto de motores gráficos, a arquitetura deve considerar separação clara entre camadas responsáveis por renderização, lógica de jogo, gerenciamento de recursos e tratamento de entrada, de modo a garantir baixo acoplamento e alta coesão entre componentes.

2.2 Padrões de Projeto e Modularidade

Os padrões de projeto constituem soluções consolidadas para problemas recorrentes no desenvolvimento de software. Segundo Gamma et al. (2000), os *design patterns* promovem reuso, organização estrutural e melhor comunicação entre desenvolvedores.

Em motores gráficos, padrões como *Singleton*, *Factory*, *Observer* e *Component* são frequentemente utilizados para estruturar subsistemas de gerenciamento global, criação de objetos, comunicação entre entidades e organização baseada em componentes. A aplicação adequada desses padrões favorece extensibilidade e manutenção, reduzindo dependência excessiva entre módulos.

A modularidade, conforme apontado por Sommerville (2019), permite dividir sistemas complexos em partes menores e independentes, tornando o desenvolvimento mais controlável e favorecendo testes isolados de funcionalidades.

2.3 Game Engines: Conceito e Estrutura

Uma *game engine* pode ser definida como um conjunto de bibliotecas e ferramentas responsáveis por fornecer infraestrutura reutilizável para o desenvolvimento de jogos digitais. Essas estruturas geralmente incluem sistemas de renderização, controle de entrada, física, áudio, gerenciamento de recursos e organização de entidades.

Motores amplamente difundidos como Unity e Unreal Engine apresentam arquiteturas generalistas, voltadas a múltiplos gêneros e plataformas. Embora ofereçam alto nível de abstração e produtividade, tais soluções podem incorporar complexidade significativa, tornando mais difícil a compreensão aprofundada de seus mecanismos internos.

Conforme Gregory (2018), motores gráficos modernos são estruturados em camadas que isolam subsistemas críticos, permitindo que o desenvolvedor concentre-se na lógica do jogo. Entretanto, a especialização para um domínio específico como jogos 2D baseados em *tiles*, pode possibilitar arquiteturas mais enxutas e direcionadas ao desempenho.

2.4 Renderização 2D e Sistemas Baseados em *Tiles*

A renderização 2D consiste na projeção de elementos gráficos bidimensionais em um plano cartesiano, organizados em camadas ou mapas. Em jogos baseados em *tiles*, o cenário é estruturado a partir de pequenas unidades gráficas repetíveis que compõem mapas maiores.

Esse modelo apresenta vantagens relacionadas à organização espacial, detecção de colisão e otimização de memória, uma vez que permite reutilização de recursos gráficos e simplificação do cálculo de posicionamento. Técnicas como *sprite batching* e controle de camadas são frequentemente utilizadas para minimizar chamadas de renderização e melhorar desempenho.

A organização em mapas discretos também facilita a implementação de sistemas de colisão baseados em grade, reduzindo complexidade computacional quando comparado a métodos contínuos.

2.5 Linguagem C++ e Desempenho

A linguagem C++ é amplamente empregada em sistemas que exigem alto desempenho e controle refinado sobre recursos computacionais. Conforme Stroustrup (2013), C++ combina abstração orientada a objetos com gerenciamento direto de memória, permitindo ao desenvolvedor equilibrar produtividade e eficiência.

No desenvolvimento de motores gráficos, o controle explícito de memória e a possibilidade de otimizações de baixo nível tornam C++ particularmente adequada. Além disso, sua compatibilidade com bibliotecas gráficas e APIs de baixo nível amplia a flexibilidade arquitetural.

O uso de C++ também favorece a implementação de bibliotecas reutilizáveis, permitindo encapsulamento de funcionalidades em módulos independentes e distribuíveis.

2.6 Gerenciamento de Recursos e Entidades

Motores gráficos necessitam administrar múltiplos recursos simultaneamente, como texturas, mapas, objetos e eventos. A organização adequada desses recursos influencia diretamente no desempenho e na estabilidade do sistema.

Abordagens modernas frequentemente adotam modelos baseados em componentes (*Entity Component System* – ECS), nos quais entidades são compostas por conjuntos de componentes independentes responsáveis por comportamentos específicos. Essa abordagem favorece flexibilidade e reduz acoplamento entre lógica e representação.

Além disso, técnicas de carregamento sob demanda e cache de recursos são estratégias comuns para evitar consumo excessivo de memória e melhorar tempo de resposta.

3 METODOLOGIA

Este trabalho caracteriza-se como uma pesquisa aplicada, uma vez que objetiva o desenvolvimento de um artefato tecnológico com finalidade prática, conforme classificação proposta por Gil (2019). Quanto aos objetivos, enquadra-se como pesquisa exploratória, pois busca aprofundar o conhecimento acerca da estrutura interna de uma *game engine* 2D e investigar soluções arquiteturais voltadas à otimização de desempenho.

No que se refere à abordagem, a pesquisa apresenta natureza predominantemente qualitativa, uma vez que se concentra na análise estrutural, organizacional e arquitetural do sistema desenvolvido, sem a aplicação de métodos estatísticos ou modelagens matemáticas formais.

3.1 Procedimentos Técnicos

Os procedimentos técnicos adotados envolveram:

- Pesquisa bibliográfica, com base em obras relacionadas à engenharia de software, arquitetura de sistemas, padrões de projeto e desenvolvimento de motores gráficos;
- Estudo conceitual sobre organização modular e gerenciamento de recursos em aplicações gráficas;
- Modelagem arquitetural do sistema antes da implementação;
- Desenvolvimento incremental da biblioteca proposta;
- Construção de protótipo funcional para validação prática.

A pesquisa bibliográfica fundamentou as decisões arquiteturais adotadas, especialmente no que se refere à modularização, baixo acoplamento e aplicação de padrões estruturais.

3.2 Modelagem Arquitetural

Antes da implementação, foi realizada a definição da arquitetura geral da *engine*, estabelecendo a separação entre os seguintes módulos principais:

- Núcleo da aplicação (*core*);
- Sistema de renderização 2D;
- Gerenciamento de recursos;
- Sistema de entidades;
- Controle de colisões;
- Interface de integração com o jogo.

Essa modelagem buscou garantir alta coesão interna e baixo acoplamento entre componentes, conforme princípios clássicos da engenharia de software (SOMMERVILLE, 2019). A definição prévia das responsabilidades de cada módulo permitiu reduzir retrabalho durante a fase de implementação.

3.3 Implementação Incremental

A implementação foi conduzida de forma incremental, priorizando inicialmente a estrutura básica da biblioteca e posteriormente adicionando funcionalidades específicas. A estratégia incremental foi adotada para permitir validação progressiva dos módulos implementados.

O desenvolvimento seguiu as seguintes etapas:

- Estruturação do núcleo da biblioteca e configuração do ambiente de compilação em C++;
- Implementação do sistema de renderização 2D voltado para mapas baseados em tiles;
- Desenvolvimento do sistema de gerenciamento de recursos gráficos;
- Implementação do sistema de entidades e lógica básica de movimentação;
- Integração entre os módulos e testes funcionais.

Essa abordagem permitiu isolamento de falhas e ajustes arquiteturais durante o processo.

3.4 Minimização de Dependências Externas

Um dos princípios orientadores do projeto foi a redução do uso de bibliotecas externas, com o objetivo de manter maior controle sobre o funcionamento interno da *engine* e aprofundar o entendimento sobre seus mecanismos estruturais.

Foram utilizadas apenas dependências estritamente necessárias para acesso a recursos de baixo nível, evitando *frameworks* de alto nível que encapsulam a lógica de renderização ou gerenciamento de entidades. Essa decisão está alinhada ao objetivo de explorar de forma direta os fundamentos técnicos envolvidos no desenvolvimento de motores gráficos.

3.5 Validação do Sistema

A validação da biblioteca foi realizada por meio do desenvolvimento de um protótipo funcional de jogo 2D baseado em *tiles*, utilizando visão *top-down*. O protótipo teve como finalidade verificar:

- Correta renderização dos mapas;
- Funcionamento do sistema de movimentação;
- Detecção básica de colisões;
- Gerenciamento adequado de recursos gráficos.

A análise dos resultados foi conduzida de forma qualitativa, observando desempenho, estabilidade e organização estrutural do código.

3.6 Limitações Metodológicas

Como o foco do trabalho está na arquitetura e implementação estrutural da *engine*, não foram realizados testes comparativos quantitativos com motores consolidados de mercado. A avaliação concentrou-se na funcionalidade, organização arquitetural e adequação ao escopo proposto.

4 PROJECT CANVAS

Fonte: Elaborado pelo autor (2026).

PROJECT CANVAS		
DESENVOLVIMENTO DE UMA GAME ENGINE 2D EM FORMATO DE BIBLIOTECA EM C++		
 PROBLEMA - Engines generalistas possuem alta complexidade estrutural. - Grande quantidade de abstrações. - Muitas dependências externas. - Dificultam o aprendizado da arquitetura de uma engine. - Pouca flexibilidade para estudos acadêmicos e projetos específicos.	 SOLUÇÃO - Desenvolvimento de uma biblioteca própria em C++. - Arquitetura modular e organizada. - Engine especializada em jogos 2D baseados em <i>tiles</i> com visão <i>top-down</i>. - Controle direto sobre renderização e recursos. - Mínima dependência de bibliotecas externas.	 PRODUTO FINAL - Biblioteca funcional compilável em C++. - Sistema de renderização 2D baseado em <i>tiles</i>. - Gerenciador de recursos gráficos. - Sistema de entidades e detecção de colisão. - Protótipo funcional de validação. - Documentação técnica e acadêmica.
 PÚBLICO-ALVO - Estudantes de Engenharia de Software. - Desenvolvedores iniciantes. - Projetos acadêmicos que demandem motor gráfico 2D simplificado. - Desenvolvedores independentes interessados em soluções especializadas.	 OBJETIVOS DO PROJETO - Desenvolver biblioteca modular e reutilizável. - Garantir funcionamento consistente do sistema de renderização 2D. - Implementar gerenciamento eficiente de recursos. - Validar a engine por meio de protótipo funcional. - Documentar arquitetura e decisões técnicas.	 ENTREGAS PRINCIPAIS - Planejamento formal do projeto. - Modelagem arquitetural da engine. - Implementação incremental da biblioteca. - Protótipo funcional de validação. - Relatórios de acompanhamento. - Trabalho acadêmico final (TCC).
 PREMISSAS - Desenvolvimento será realizado individualmente. - A linguagem principal utilizada será C++. - O projeto será executado dentro do prazo acadêmico estabelecido. - Ambiente computacional necessário estará disponível.	 RESTRIÇÕES - Tempo limitado ao período letivo. - Recursos financeiros inexistentes. - Escopo restrito a jogos 2D baseados em <i>tiles</i> com visão <i>top-down</i>. - Complexidade técnica inerente ao desenvolvimento de motor gráfico.	 RISCOS INICIAIS - Possível atraso na implementação devido à complexidade técnica. - Dificuldade de integração entre módulos. - Problemas de desempenho inesperados. - Sobrecarga de atividades acadêmicas paralelas.

4.1 Descrição do Project Canvas

O Project Canvas apresentado na Figura sintetiza os principais elementos estratégicos do projeto, permitindo uma visão estruturada dos objetivos, entregas, restrições e riscos envolvidos no desenvolvimento da game engine 2D em C++.

4.1.1 Problema

Desenvolvedores iniciantes e projetos acadêmicos que buscam criar jogos 2D baseados em *tiles* frequentemente utilizam motores generalistas que apresentam elevada complexidade estrutural, abstrações amplas e múltiplas dependências externas. Essa característica pode dificultar a compreensão aprofundada dos mecanismos internos de uma *engine* e limitar a experimentação arquitetural voltada ao desempenho e à modularidade.

4.1.2 Solução Proposta

Desenvolver uma game engine 2D estruturada como biblioteca em C++, especializada em jogos baseados em *tiles* com visão *top-down*, priorizando arquitetura modular, controle direto sobre recursos computacionais e redução de dependências externas.

4.1.3 Produto Final

O produto final do projeto consiste em:

- Biblioteca funcional compilável em C++;
- Sistema de renderização 2D baseado em *tiles*;
- Gerenciador de recursos gráficos;
- Sistema de entidades e detecção de colisão;
- Protótipo funcional de validação;
- Documentação técnica e acadêmica.

4.1.4 Público-Alvo

O projeto é direcionado a:

- Estudantes de Engenharia de Software;
- Desenvolvedores iniciantes;
- Projetos acadêmicos que demandem motor gráfico 2D simplificado;
- Desenvolvedores independentes interessados em soluções especializadas.

4.1.5 Objetivos do Projeto

- Desenvolver biblioteca modular e reutilizável;
- Garantir funcionamento consistente do sistema de renderização 2D;
- Implementar gerenciamento eficiente de recursos;
- Validar a *engine* por meio de protótipo funcional;
- Documentar arquitetura e decisões técnicas.

4.1.6 Entregas Principais

As entregas previstas incluem:

- Planejamento formal do projeto;
- Modelagem arquitetural;
- Implementação incremental da biblioteca;
- Protótipo funcional;
- Relatórios de acompanhamento;
- Trabalho acadêmico final.

4.1.7 Premissas

- Desenvolvimento será realizado individualmente;
- A linguagem principal utilizada será C++;
- Projeto será executado dentro do prazo acadêmico estabelecido;
- Ambiente computacional necessário estará disponível.

4.1.8 Restrições

- Tempo limitado ao período letivo;
- Recursos financeiros inexistentes;
- Escopo delimitado a jogos 2D baseados em *tiles*;
- Complexidade técnica inerente ao desenvolvimento de motor gráfico.

4.1.9 Riscos Iniciais

- Possível atraso na implementação devido à complexidade técnica;
- Dificuldade de integração entre módulos;
- Problemas de desempenho inesperados;
- Sobrecarga de atividades acadêmicas paralelas.

5 GERENCIAMENTO DO ESCOPO

O gerenciamento do escopo tem como finalidade definir claramente os limites do projeto, estabelecendo o que será desenvolvido, quais entregas serão realizadas e quais elementos não fazem parte da proposta. Essa definição busca evitar expansão não controlada do projeto (*scope creep*), assegurando foco nos objetivos previamente estabelecidos.

5.1 Objetivos SMART

Os objetivos do projeto foram definidos segundo o modelo SMART, garantindo que sejam específicos, mensuráveis, alcançáveis, relevantes e temporais.

Objetivo Geral SMART

Desenvolver, até o final do período letivo de 2026-1, uma *game engine* 2D em formato de biblioteca, utilizando a linguagem C++, contendo sistema de renderização baseado em *tiles*, gerenciamento de recursos, sistema de entidades e mecanismo básico de colisão, validada por meio de protótipo funcional.

Especificidade (S – *Specific*)

O projeto consiste na criação de uma biblioteca especializada para jogos 2D com visão *top-down*, não abrangendo funcionalidades de motores generalistas completos.

Mensurabilidade (M – *Measurable*)

O sucesso do projeto será verificado por meio de:

- Compilação funcional da biblioteca;
- Execução do protótipo com renderização correta de *tiles*;
- Funcionamento consistente do sistema de entidades e colisão;
- Entrega da documentação técnica e acadêmica.

Atingibilidade (A – *Achievable*)

O projeto é viável considerando:

- Desenvolvimento individual;

- Escopo delimitado;
- Uso de ferramentas já dominadas pelo autor;
- Planejamento incremental das atividades.

Relevância (R – *Relevant*)

O projeto é relevante por:

- Aplicar conceitos de engenharia de software;
- Permitir aprofundamento em arquitetura e desempenho;
- Contribuir para formação acadêmica e técnica do autor.

Temporalidade (T – *Time-bound*)

O projeto será concluído dentro do cronograma acadêmico estabelecido para o semestre 2027-1.

5.2 Produto e Entregas

Produto Final

Biblioteca de game *engine* 2D implementada em C++, estruturada modularmente e validada por protótipo funcional.

Entregas Principais

1. Plano de Gerenciamento do Projeto.
2. Modelagem arquitetural da *engine*.
3. Implementação do núcleo da biblioteca.
4. Sistema de renderização baseado em *tiles*.
5. Sistema de gerenciamento de recursos.

6. Sistema de entidades e colisão.
7. Protótipo funcional de validação.
8. Documentação técnica.
9. Trabalho acadêmico final.

5.3 Matriz de Rastreabilidade dos Requisitos

A matriz de rastreabilidade tem como objetivo garantir que cada requisito identificado esteja vinculado a uma entrega específica do projeto, permitindo controle e verificação de conformidade.

ID	Requisito	Tipo	Entrega Relacionada	Critério de Validação
RF01	Renderizar mapa baseado em <i>tiles</i>	Funcional	Renderização	Execução correta no protótipo
RF02	Gerenciar carregamento de texturas	Funcional	<i>Resource Manager</i>	Carregamento sem duplicação
RF03	Criar e atualizar entidades	Funcional	<i>Entity System</i>	Atualização correta em execução
RF04	Detectar colisão com mapa	Funcional	<i>Collision System</i>	Bloqueio em áreas não transitáveis
RNF01	Manter arquitetura modular	Não funcional	Estrutura da biblioteca	Separação clara de módulos
RNF02	Minimizar dependências externas	Não funcional	Implementação geral	Uso restrito a bibliotecas essenciais

5.4 Premissas

- O desenvolvimento será realizado individualmente;
- O ambiente computacional será suficiente para execução do projeto;
- O autor possui conhecimento prévio em C++;
- O prazo acadêmico será mantido conforme calendário institucional.

5.5 Restrições

- Prazo limitado ao semestre letivo;
- Escopo restrito a jogos 2D baseados em *tiles*;
- Recursos financeiros inexistentes;
- Desenvolvimento sem equipe auxiliar.

5.6 Estrutura Analítica do Projeto (EAP)

A Estrutura Analítica do Projeto organiza o trabalho em níveis hierárquicos, facilitando planejamento e controle.

Nível 1 – Projeto *Game Engine* 2D

1. Planejamento
2. Modelagem Arquitetural
3. Implementação
 - 3.1 Núcleo (*Core*)
 - 3.2 Renderização 2D
 - 3.3 Gerenciamento de Recursos
 - 3.4 Sistema de Entidades
 - 3.5 Sistema de Colisão
4. Testes e Validação
5. Documentação
6. Entrega Final

5.7 Principais Atividades e Estratégia do Projeto

O projeto será conduzido por meio de desenvolvimento incremental, priorizando inicialmente a estrutura básica da biblioteca e posteriormente adicionando funcionalidades específicas.

A estratégia adotada inclui:

- Definição arquitetural prévia;
- Implementação modular;
- Validação progressiva por protótipo;
- Revisões periódicas de progresso;
- Documentação paralela ao desenvolvimento.

Essa abordagem busca reduzir riscos técnicos e permitir ajustes estruturais durante a execução do projeto.

6 GERENCIAMENTO DO TEMPO

O gerenciamento do tempo tem como objetivo planejar, estimar, organizar e controlar a duração das atividades do projeto, assegurando que a *game engine* 2D seja desenvolvida dentro do prazo acadêmico estabelecido para o semestre 2027-1.

O planejamento temporal foi estruturado considerando:

- Complexidade técnica das etapas;
- Desenvolvimento individual;
- Necessidade de paralelismo entre implementação e documentação;
- Períodos de revisão e ajustes.

6.1 Definição das Atividades

As atividades do projeto foram estruturadas a partir da EAP apresentada anteriormente.

Fase 1 – Planejamento

- Elaboração do Plano de Gerenciamento
- Definição da arquitetura inicial
- Levantamento de requisitos

Fase 2 – Modelagem Arquitetural

- Modelagem UML (classes e módulos)
- Definição de responsabilidades
- Estruturação dos pacotes da biblioteca

Fase 3 – Implementação do Núcleo

- Estrutura base do projeto
- *Loop* principal
- Sistema de inicialização

Fase 4 – Renderização 2D

- Sistema de *tiles*
- Carregamento de *sprites*
- Renderização em tela

Fase 5 – Gerenciamento de Recursos

- Implementação do *Resource Manager*
- Controle de carregamento e descarregamento
- Otimização de memória

Fase 6 – Sistema de Entidades

- Criação da classe base *Entity*
- Sistema de atualização
- Gerenciamento de múltiplas entidades

Fase 7 – Sistema de Colisão

- Implementação de *bounding box*
- Verificação de colisão com mapa
- Integração com entidades

Fase 8 – Testes e Validação

- Criação do protótipo funcional
- Testes de desempenho
- Correção de inconsistências

Fase 9 – Documentação Final

- Revisão do TCC
- Consolidação de diagramas
- Preparação para entrega

6.2 Sequenciamento das Atividades

As atividades seguem ordem lógica de dependência técnica:

1. Planejamento
2. Modelagem
3. Implementação do Núcleo
4. Renderização
5. Recursos

- 6. Entidades
- 7. Colisão
- 8. Testes
- 9. Documentação Final

A implementação ocorre de forma incremental, permitindo validação contínua.

6.3 Estimativa de Duração

A estimativa foi realizada considerando semanas acadêmicas.

Fase	Atividade	Duração
1	Planejamento	2 semanas
2	Modelagem	2 semanas
3	Núcleo	3 semanas
4	Renderização	3 semanas
5	Recursos	2 semanas
6	Entidades	2 semanas
7	Colisão	2 semanas
8	Testes	2 semanas
9	Documentação	2 semanas

Duração total estimada: aproximadamente 20 semanas.

6.4 Marcos do Projeto

Os marcos representam pontos críticos de verificação:

- M1 – Aprovação do Plano de Gerenciamento
- M2 – Conclusão da Modelagem Arquitetural
- M3 – Renderização funcional de mapa
- M4 – Sistema de entidades operacional
- M5 – Protótipo completo executando
- M6 – Entrega do TCC

6.5 Cronograma (Modelo para Gráfico de Gantt)

Fonte: Elaborado pelo autor (2026).

Atividade	Semanas																			
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
Planejamento	X	X																		
Modelagem			X	X																
Núcleo					X	X	X													
Renderização								X	X	X										
Recursos											X	X								
Entidades													X	X						
Colisão															X	X				
Testes																	X	X		
Documentação																			X	X

6.6 Controle do Cronograma

O acompanhamento será realizado por meio de:

- Revisões semanais de progresso;

- Comparação entre prazo planejado e prazo real;
- Ajustes pontuais de prioridade quando necessário;
- Registro de atrasos e suas causas.

Caso haja risco de atraso, será adotada reordenação de atividades ou simplificação de funcionalidades não críticas.

7 EQUIPE

O gerenciamento da equipe tem como objetivo definir responsabilidades, papéis, formas de comunicação e mecanismos de acompanhamento das atividades do projeto. Embora o desenvolvimento da *game engine* 2D seja realizado individualmente, a formalização da estrutura organizacional contribui para maior clareza na execução das tarefas e na prestação de contas acadêmica.

7.1 Estrutura Organizacional do Projeto

O projeto possui estrutura organizacional simplificada, caracterizada por modelo funcional individual, no qual o autor desempenha múltiplos papéis técnicos e gerenciais.

Papéis Identificados

Papel	Responsável	Responsabilidades
Gerente do Projeto	Autor do projeto	Planejamento, controle de escopo, prazo e riscos
Arquiteto de Software	Autor do projeto	Definição da arquitetura e modularização
Desenvolvedor	Autor do projeto	Implementação da biblioteca
Analista de Testes	Autor do projeto	Validação e verificação funcional

Documentador Técnico	Autor do projeto	Produção da documentação acadêmica
Orientador Acadêmico	Professor da disciplina	Acompanhamento, validação metodológica e avaliação

7.2 Responsabilidades

Autor do Projeto

- Elaborar o planejamento formal;
- Desenvolver a arquitetura da *engine*;
- Implementar os módulos previstos;
- Realizar testes funcionais;
- Produzir documentação técnica e acadêmica;
- Monitorar prazos e riscos.

Orientador Acadêmico

- Avaliar coerência metodológica;
- Validar entregas parciais;
- Fornecer direcionamento acadêmico;
- Avaliar o resultado final.

7.3 Matriz de Responsabilidades (RACI Simplificada)

Atividade	Autor do projeto	Orientador Acadêmico
Planejamento	R	A
Modelagem	R	C
Implementação	R	I
Testes	R	I
Documentação	R	C

Aprovação Final	-	A

Legenda:

R – Responsável

A – Aprovador

C – Consultado

I – Informado

7.4 Competências Necessárias

Para execução do projeto são requeridas as seguintes competências:

- Programação avançada em C++;
- Conhecimento em arquitetura de software;
- Modelagem UML;
- Noções de desempenho e gerenciamento de memória;
- Planejamento e organização de projetos;
- Escrita acadêmica conforme normas técnicas.

7.5 Estratégia de Desenvolvimento Individual

Considerando que o projeto é desenvolvido individualmente, será adotada estratégia de organização baseada em:

- Planejamento semanal de tarefas;
- Registro de progresso;
- Revisões periódicas de arquitetura;
- Desenvolvimento incremental;
- Testes contínuos após cada módulo implementado.

Essa abordagem busca minimizar riscos relacionados à sobrecarga de atividades e atrasos no cronograma.

7.6 Plano de Comunicação Interna

Mesmo sendo projeto individual, o controle será realizado por meio de:

- Registro de decisões técnicas;
- Organização de versões do código;
- Atualização periódica do documento acadêmico;
- Reuniões de acompanhamento com orientador.

8 GERENCIAMENTO DAS COMUNICAÇÕES

O gerenciamento das comunicações tem como finalidade assegurar que todas as informações relevantes do projeto sejam geradas, registradas, armazenadas e transmitidas de maneira adequada às partes interessadas, especialmente ao orientador acadêmico.

Este plano estabelece os canais, frequência e responsabilidades relacionados à comunicação durante o desenvolvimento da *game engine* 2D.

8.1 Objetivos da Comunicação

- Garantir alinhamento entre autor e orientador;
- Registrar decisões técnicas relevantes;
- Documentar progresso do projeto;

- Facilitar acompanhamento acadêmico;
- Manter histórico das versões e alterações realizadas.

8.2 Partes Interessadas na Comunicação

Parte Interessada	Interesse	Necessidade de Informação
Autor do projeto	Execução do projeto	Planejamento, progresso, riscos
Orientador acadêmico	Acompanhamento acadêmico	Status, entregas parciais, dificuldades
Instituição	Avaliação formal	Entrega final e documentação

8.3 Canais de Comunicação

Os seguintes canais serão utilizados:

- Reuniões presenciais ou virtuais com o orientador;
- Comunicação por e-mail institucional;
- Registro escrito no próprio documento do projeto;
- Controle de versões do código-fonte;
- Armazenamento digital organizado.

8.4 Frequência das Comunicações

Tipo de Comunicação	Frequência	Responsável
Reunião de acompanhamento	Quinzenal	Autor e Orientador
Atualização de progresso	Semanal	Autor do projeto
Entrega parcial de documentos	Conforme cronograma	Autor do projeto
<i>Feedback</i> técnico/metodológico	Conforme necessidade	Orientador Acadêmico

8.5 Registro e Armazenamento de Informações

Serão adotadas as seguintes práticas:

- Versionamento organizado do código-fonte;
- Armazenamento digital dos documentos em pasta estruturada;
- Registro de alterações significativas no projeto;
- *Backup* periódico dos arquivos.

Essa organização busca garantir segurança das informações e rastreabilidade das decisões técnicas.

8.6 Tipos de Informação Comunicada

Durante o projeto, serão comunicadas:

- Alterações no escopo;
- Ajustes no cronograma;
- Identificação de riscos;
- Dificuldades técnicas;
- Conclusão de marcos importantes.

8.7 Fluxo de Comunicação

O fluxo comunicacional do projeto segue estrutura simples:

1. Autor do projeto
2. Registro formal
3. Orientador acadêmico

4. *Feedback*
5. Ajustes no projeto

Esse fluxo assegura que decisões relevantes sejam validadas e documentadas.

9 GERENCIAMENTO DAS PARTES INTERESSADAS

O gerenciamento das partes interessadas tem como objetivo identificar indivíduos ou grupos que possam influenciar ou ser impactados pelo projeto, analisando seu nível de interesse e influência, bem como definindo estratégias adequadas de engajamento.

No contexto deste projeto, as partes interessadas estão relacionadas principalmente ao ambiente acadêmico e ao próprio autor.

9.1 Identificação das Partes Interessadas

As principais partes interessadas do projeto são:

- Autor do projeto
- Orientador acadêmico
- Instituição de ensino
- Banca avaliadora
- Comunidade acadêmica e desenvolvedores interessados

9.2 Análise das Partes Interessadas

A análise considera dois fatores principais: nível de interesse e nível de influência.

Parte Interessada	Interesse	Influência	Classificação
Autor do projeto	Muito alto	Alto	Principal interessado
Orientador acadêmico	Alto	Alto	Influenciador direto
Instituição	Médio	Alto	Regulador formal
Banca avaliadora	Alto	Alto	Avaliador final
Comunidade técnica	Médio	Baixo	Beneficiário indireto

9.3 Matriz Interesse x Influência

A partir da análise, estabelece-se a seguinte estratégia:

- Alta influência / Alto interesse → Gerenciar de perto
 - o Orientador acadêmico
 - o Banca avaliadora
- Alta influência / Médio interesse → Manter satisfeito
 - o Instituição
- Alto interesse / Alta responsabilidade → Controle direto
 - o Autor do projeto
- Baixa influência / Médio interesse → Monitorar
 - o Comunidade técnica

9.4 Estratégias de Engajamento

Autor do projeto

- Planejamento estruturado;
- Registro contínuo de progresso;
- Controle disciplinado do cronograma.

Orientador acadêmico

- Reuniões periódicas;
- Entregas parciais para validação;
- Ajustes conforme orientação metodológica.

Instituição

- Atendimento às normas acadêmicas;
- Cumprimento dos prazos estabelecidos.

Banca Avaliadora

- Produção de documento consistente;
- Clareza técnica na defesa do projeto.

Comunidade Técnica

- Possível disponibilização futura da biblioteca;
- Documentação clara e organizada.

9.5 Monitoramento das Partes Interessadas

O acompanhamento será realizado por meio de:

- *Feedback* do orientador;
- Cumprimento dos marcos acadêmicos;
- Avaliação contínua do alinhamento do projeto com os objetivos institucionais.

Caso haja alteração no nível de interesse ou influência de alguma parte interessada, o plano poderá ser ajustado.

10 GERENCIAMENTO DOS RISCOS

O gerenciamento dos riscos tem como objetivo identificar, analisar e propor respostas para eventos que possam impactar negativamente o desempenho do projeto, seja em termos de prazo, escopo ou qualidade.

Considerando que o desenvolvimento da *game engine* 2D é realizado individualmente e envolve complexidade técnica relevante, a gestão de riscos torna-se fundamental para garantir a viabilidade do projeto.

10.1 Identificação dos Riscos

Os principais riscos identificados para o projeto são:

- Complexidade técnica superior à estimada;
- Atraso no cronograma;
- Problemas de desempenho da biblioteca;
- Dificuldades na integração entre módulos;
- Sobrecarga acadêmica;
- Perda de dados ou falhas no ambiente de desenvolvimento.

10.2 Análise Qualitativa dos Riscos

A análise qualitativa considera probabilidade e impacto.

ID	Risco	Probabilidade	Impacto	Nível
R1	Complexidade técnica elevada	Média	Alta	Alto
R2	Atraso no cronograma	Média	Alta	Alto
R3	Baixo desempenho da <i>engine</i>	Baixa	Alta	Média
R4	Falha na integração de módulos	Média	Média	Média
R5	Sobrecarga académica	Alta	Média	Alto
R6	Perda de arquivos	Baixa	Alta	Média

10.3 Estratégias de Resposta aos Riscos

R1 – Complexidade Técnica Elevada

Estratégia: Mitigação

- Dividir implementação em etapas menores;
- Validar cada módulo isoladamente;
- Realizar testes incrementais.

R2 – Atraso no Cronograma

Estratégia: Mitigação

- Planejamento semanal;
- Priorização de funcionalidades essenciais;
- Redução de funcionalidades não críticas, se necessário.

R3 – Baixo Desempenho

Estratégia: Mitigação

- Testes de desempenho periódicos;
- Revisão de código e otimização de memória.

R4 – Falha na Integração

Estratégia: Mitigação

- Definição clara de *interfaces*;
- Testes de integração progressivos.

R5 – Sobrecarga Acadêmica

Estratégia: Aceitação controlada com monitoramento

- Organização antecipada de tarefas;
- Ajustes no cronograma quando necessário.

R6 – Perda de Arquivos

Estratégia: Prevenção

- Backup periódico;
- Versionamento do código.

10.4 Plano de Contingência

Caso ocorra materialização de riscos críticos:

- Redução do escopo não essencial;
- Reorganização do cronograma;
- Simplificação de funcionalidades avançadas;
- Priorização da estabilidade do núcleo da biblioteca.

10.5 Monitoramento dos Riscos

O acompanhamento será realizado por meio de:

- Revisão semanal do progresso;
- Atualização do status dos riscos identificados;
- Registro de novos riscos que surgirem durante o desenvolvimento.

O plano de riscos poderá ser revisado sempre que houver alteração significativa no escopo ou cronograma.

11 MONITORAMENTO E CONTROLE

O monitoramento e controle do projeto têm como objetivo acompanhar o desempenho das atividades planejadas, verificando desvios em relação ao escopo, prazo e qualidade, e promovendo ajustes quando necessário.

No contexto do desenvolvimento da *game engine* 2D em C++, o controle será realizado de forma contínua, com foco na disciplina organizacional e na validação incremental das entregas.

11.1 Controle do Escopo

O controle do escopo será realizado por meio de:

- Verificação periódica dos requisitos definidos;
- Conferência entre funcionalidades implementadas e entregas previstas;
- Registro formal de qualquer alteração no escopo;
- Avaliação de impacto antes da inclusão de novas funcionalidades.

Qualquer solicitação de modificação será analisada quanto a:

- Impacto no prazo;
- Impacto na complexidade técnica;
- Necessidade real para os objetivos do projeto.

11.2 Controle do Cronograma

O acompanhamento do prazo será realizado por meio de:

- Revisão semanal das atividades planejadas;
- Comparação entre duração estimada e duração real;
- Identificação antecipada de atrasos;
- Replanejamento quando necessário.

Em caso de desvio significativo, poderão ser adotadas medidas como:

- Priorização de funcionalidades essenciais;
- Simplificação de módulos secundários;
- Ajuste na sequência das atividades.

11.3 Controle da Qualidade

A qualidade do projeto será monitorada por meio de:

- Testes funcionais após implementação de cada módulo;
- Execução do protótipo de validação;
- Revisão de código;
- Verificação da aderência à arquitetura definida;
- Conferência da documentação técnica.

Critérios de qualidade incluem:

- Funcionamento correto do sistema de renderização;
- Estabilidade da execução;
- Organização modular do código;
- Clareza da documentação.

11.4 Indicadores de Desempenho

Serão utilizados os seguintes indicadores:

- Percentual de atividades concluídas no prazo;
- Número de funcionalidades implementadas conforme requisitos;
- Número de erros identificados durante testes;
- Cumprimento dos marcos do projeto.

11.5 Relatórios de Acompanhamento

O acompanhamento do projeto será documentado por meio de:

- Atualizações semanais de progresso;
- Registro de dificuldades técnicas;
- Atualização da matriz de riscos;
- Consolidação de marcos atingidos.

Esses registros servirão como base para análise do desempenho do projeto ao final do período acadêmico.

11.6 Processo de Ajustes

Caso sejam identificados desvios relevantes:

- problema será analisado;
- Será avaliado o impacto no escopo e no prazo;
- Definir-se-á uma ação corretiva;
- ajuste será registrado formalmente.

Essa sistemática garante controle estruturado e reduz a probabilidade de decisões improvisadas.

12 CONTROLE INTEGRADO DE MUDANÇAS

O controle integrado de mudanças tem como finalidade assegurar que qualquer alteração no escopo, cronograma, requisitos ou arquitetura do projeto seja analisada de forma estruturada antes de sua implementação.

Considerando que o desenvolvimento da *game engine* 2D envolve decisões técnicas contínuas, este processo busca evitar modificações impulsivas que possam comprometer prazo, estabilidade ou coerência arquitetural.

12.1 Objetivo do Controle de Mudanças

- Garantir que alterações sejam justificadas;
- Avaliar impacto técnico e temporal;
- Manter rastreabilidade das decisões;
- Preservar alinhamento com os objetivos do projeto.

12.2 Tipos de Mudanças Possíveis

As mudanças podem envolver:

- Inclusão de novas funcionalidades;
- Remoção de funcionalidades previstas;
- Alterações na arquitetura;
- Ajustes no cronograma;
- Modificações nos requisitos funcionais ou não funcionais.

12.3 Processo de Solicitação de Mudança

Toda mudança deverá seguir as seguintes etapas:

1. Identificação da necessidade de alteração;
2. Registro da solicitação de mudança;
3. Análise de impacto (escopo, prazo e risco);
4. Decisão de aprovação ou rejeição;
5. Atualização formal do plano e documentação.

12.4 Critérios para Aprovação de Mudanças

Uma mudança poderá ser aprovada quando:

- Contribuir diretamente para o objetivo principal do projeto;
- Não comprometer o prazo final;
- Não aumentar significativamente o risco;
- For tecnicamente viável dentro do tempo disponível.

Mudanças que representem ampliação significativa de escopo poderão ser adiadas ou descartadas.

12.5 Registro das Mudanças

Será mantido um controle simplificado de mudanças conforme o modelo:

Durante o desenvolvimento da *engine* foram identificadas oportunidades de melhoria arquitetural, correções funcionais e novas funcionalidades não previstas inicialmente. Todas as alterações relevantes foram registradas para manter a rastreabilidade das decisões técnicas adotadas e seus impactos sobre o projeto.

ID	Descrição da Mudança	Justificativa	Impacto	Decisão
M01	Definição da arquitetura em módulos (<i>Core, Renderer, Input, Scene, Entity, Physics, Graphics, World, ResourceManager</i>).	Facilitar manutenção, organização e escalabilidade da <i>engine</i> .	Alto	Aprovada
M02	Implementação do sistema de gerenciamento de cenas (<i>MainMenu, Game, Pause, Options</i> e <i>GameOver</i>).	Separar responsabilidades e permitir troca dinâmica de estados do jogo.	Alto	Aprovada
M03	Criação do sistema de entidades utilizando herança (<i>Entity, Player</i> e <i>Enemy</i>).	Padronizar o comportamento dos objetos do jogo.	Alto	Aprovada
M04	Implementação do <i>ResourceManager</i> .	Evitar carregamento duplicado de texturas e centralizar o gerenciamento de recursos.	Médio	Aprovada
M05	Desenvolvimento do sistema de <i>sprites</i> e texturas utilizando <i>SDL2_image</i> .	Permitir renderização de imagens na <i>engine</i> .	Alto	Aprovada
M06	Implementação do sistema de renderização de texto utilizando <i>SDL2_ttf</i> .	Possibilitar criação de menus e interface gráfica.	Médio	Aprovada
M07	Inclusão do <i>TileMap</i> baseado em arquivos externos (.txt).	Separar os mapas do código-fonte e facilitar futuras edições.	Alto	Aprovada
M08	Implementação do sistema de colisão com o <i>TileMap</i> .	Impedir movimentação em paredes e delimitar a	Alto	Aprovada

		navegação do jogador.		
M09	Ajuste da colisão considerando toda a área do <i>Collider</i> .	Corrigir pequenas invasões do jogador nas paredes do mapa.	Médio	Aprovada
M10	Criação do sistema de câmera com acompanhamento do jogador.	Permitir mapas maiores que a área visível da janela.	Alto	Aprovada
M11	Separação entre renderização da cena e renderização da interface (UI).	Evitar que menus fossem deslocados pela movimentação da câmera.	Médio	Aprovada
M12	Padronização da estrutura de diretórios do projeto (<i>assets</i> , <i>include</i> , <i>src</i> , <i>docs</i> e <i>tests</i>).	Melhorar organização e facilitar manutenção.	Médio	Aprovada
M13	Migração do gerenciamento de memória para <code>std::unique_ptr</code> .	Eliminar gerenciamento manual de memória e reduzir risco de vazamentos.	Alto	Aprovada
M14	Implementação do sistema de entrada utilizando <i>InputManager</i> .	Centralizar o tratamento dos eventos de teclado.	Médio	Aprovada
M15	Ajustes na estrutura do CMake e organização das dependências SDL2.	Melhorar a portabilidade e simplificar a compilação da <i>engine</i> .	Médio	Aprovada

O registro das mudanças permitiu documentar as decisões técnicas adotadas, justificando cada alteração e seus respectivos impactos no escopo e na implementação.

12.6 Autoridade de Aprovação

Como o projeto é individual, o autor atua como responsável pela análise técnica das mudanças. Entretanto, alterações que impactem o escopo acadêmico deverão ser discutidas com o orientador antes da implementação.

12.7 Controle de Versões

O controle de mudanças será complementado por:

- Versionamento organizado do código;

- Identificação clara das versões da biblioteca;
- Registro das principais alterações realizadas em cada etapa.

Com isso, o projeto passa a ter um mecanismo formal de governança técnica, reduzindo riscos de desorganização ou expansão não planejada do escopo.

13 LIÇÕES APRENDIDAS

O desenvolvimento da engine 2D em C++ proporcionou diversos aprendizados técnicos e de engenharia de software. Ao longo do projeto foram enfrentados desafios relacionados à arquitetura, gerenciamento de recursos, renderização, movimentação e organização do código. As principais lições aprendidas foram:

1. A definição de uma arquitetura modular desde o início facilita significativamente a evolução do projeto. A separação em módulos como *Core*, *Renderer*, *Input*, *Scene*, *World*, *Entity*, *Graphics* e *Physics* reduziu o acoplamento entre os componentes e tornou a manutenção mais simples durante a implementação de novas funcionalidades.
2. O gerenciamento automático de memória utilizando `std::unique_ptr` tornou a *engine* mais segura. A substituição do gerenciamento manual evitou vazamentos de memória e simplificou o controle do ciclo de vida das entidades e das cenas.
3. Centralizar o carregamento de recursos melhorou a organização da aplicação. A implementação do *ResourceManager* evitou o carregamento duplicado de texturas e criou um ponto único para gerenciamento dos recursos gráficos da *engine*.
4. A implementação da colisão baseada em tiles exigiu diversos refinamentos. Inicialmente a verificação utilizava apenas um ponto do jogador, permitindo

pequenas invasões nas paredes. A solução foi adaptar o algoritmo para considerar toda a área do *Collider*, tornando a colisão mais precisa.

5. A implementação da câmera evidenciou a necessidade de separar a renderização do mundo da interface gráfica. Após a câmera passar a acompanhar o jogador, elementos da *interface* começaram a sofrer deslocamentos indevidos. Esse problema foi resolvido criando métodos específicos para renderização da interface (UI), independentes da câmera.
6. A utilização de mapas externos em arquivos texto tornou o desenvolvimento mais flexível. A criação do sistema *TileMap* permitiu modificar mapas sem necessidade de recompilar a aplicação, facilitando testes, criação de fases e manutenção futura.
7. A organização do projeto em diretórios específicos aumentou a produtividade durante o desenvolvimento. A separação entre código-fonte, cabeçalhos, recursos gráficos, mapas, documentação e testes tornou a estrutura mais clara e preparada para expansão.
8. A implementação do sistema de cenas demonstrou a importância da separação de responsabilidades. O gerenciamento independente das telas de Menu Principal, Opções, Jogo, Pausa e *Game Over* simplificou a navegação entre estados da aplicação e reduziu o impacto de alterações futuras.
9. A construção incremental da engine facilitou a identificação e correção de problemas. Cada funcionalidade foi implementada, testada e validada antes da introdução da próxima, permitindo localizar rapidamente erros de compilação, renderização e lógica durante o desenvolvimento.
10. O uso do CMake contribuiu para a padronização do processo de compilação. A automatização da configuração do projeto simplificou o gerenciamento das

dependências da SDL2 e tornou a compilação mais organizada em diferentes ambientes.

11. A criação de uma API própria para a engine reforçou a importância do encapsulamento. Métodos públicos como movimentação, renderização, gerenciamento de entidades e acesso aos recursos permitiram ocultar detalhes internos da implementação, tornando a *engine* mais simples de utilizar.
12. A documentação produzida durante o desenvolvimento facilitou a evolução do projeto. A manutenção da documentação técnica paralelamente à implementação permitiu registrar decisões arquiteturais, justificar alterações e organizar a evolução da *engine*.
13. O projeto demonstrou que uma arquitetura bem planejada facilita futuras expansões. A estrutura atual permite incorporar novos módulos, como animações, áudio, inteligência artificial, sistema de partículas, *scripting* e editor de mapas, preservando a organização existente e reduzindo o impacto sobre os componentes já implementados.

14 REQUISITOS ESPECÍFICOS

Esta seção apresenta os requisitos específicos do sistema, divididos em requisitos funcionais e requisitos não funcionais. Os requisitos funcionais descrevem as funcionalidades que a *engine* deve fornecer ao usuário, enquanto os requisitos não funcionais estabelecem critérios de qualidade e restrições que devem ser atendidas pelo sistema.

14.1 Requisitos Funcionais

Os requisitos funcionais descrevem as operações e serviços que o sistema deve oferecer aos usuários e desenvolvedores que utilizarem a *engine*.

RF01 – Inicialização da *Engine*

A *engine* deve permitir a inicialização do sistema através de uma interface de programação que configure os componentes principais necessários para execução do jogo.

RF02 – Gerenciamento do *Loop* Principal

A *engine* deve fornecer um mecanismo responsável por controlar o *loop* principal do jogo, incluindo atualização lógica, processamento de entrada e renderização gráfica.

RF03 – Renderização de Elementos Gráficos

A *engine* deve permitir a renderização de *sprites* e outros elementos gráficos bidimensionais na tela.

RF04 – Gerenciamento de Entidades

A *engine* deve permitir a criação, atualização e remoção de entidades dentro do ambiente do jogo.

RF05 – Sistema de Colisão

A *engine* deve fornecer mecanismos para detecção de colisões entre entidades presentes no ambiente do jogo.

RF06 – Gerenciamento de Recursos

A *engine* deve permitir o carregamento e gerenciamento de recursos utilizados pelo jogo, como imagens e arquivos de mapas.

RF07 – Sistema de Entrada do Usuário

A *engine* deve capturar e processar eventos de entrada do usuário provenientes de dispositivos como teclado e mouse.

RF08 – Estrutura de Cenas

A *engine* deve permitir a organização do jogo em diferentes cenas ou estados, possibilitando a transição entre diferentes contextos do jogo.

RF09 – Atualização de Entidades

A *engine* deve permitir que entidades sejam atualizadas a cada ciclo do loop do jogo, possibilitando o processamento de lógica e comportamento.

RF10 – Encerramento do Sistema

A *engine* deve fornecer mecanismos para encerramento seguro da execução do jogo e liberação adequada dos recursos utilizados.

14.2 Requisitos Não Funcionais

Os requisitos não funcionais definem atributos de qualidade e restrições técnicas que o sistema deve atender durante sua execução e manutenção.

RNF01 – Desempenho

A *engine* deve ser capaz de manter uma taxa mínima de 60 quadros por segundo em condições normais de execução.

RNF02 – Modularidade

A arquitetura da *engine* deve ser organizada de forma modular, permitindo que diferentes componentes do sistema sejam desenvolvidos e mantidos de forma independente.

RNF03 – Manutenibilidade

O código-fonte do sistema deve seguir boas práticas de organização e documentação, facilitando sua manutenção e evolução.

RNF04 – Portabilidade

A *engine* deve ser desenvolvida de forma que possa ser executada em diferentes sistemas operacionais compatíveis com compiladores C++ modernos.

RNF05 – Escalabilidade

A arquitetura do sistema deve permitir a adição de novos módulos e funcionalidades sem necessidade de alterações estruturais significativas.

RNF06 – Usabilidade da API

A API da *engine* deve apresentar uma interface clara e consistente, facilitando sua utilização por desenvolvedores.

RNF07 – Organização do Código

O código da *engine* deve ser estruturado em módulos ou componentes claramente definidos, com responsabilidades bem delimitadas.

15 VISÃO GERAL DA ARQUITETURA

A arquitetura da *engine* foi projetada com o objetivo de fornecer uma estrutura modular e organizada para o desenvolvimento de jogos bidimensionais utilizando a linguagem de programação C++. Essa abordagem permite que diferentes componentes do sistema sejam desenvolvidos e mantidos de forma independente, favorecendo a organização do código e facilitando futuras extensões.

A *engine* é composta por um conjunto de módulos responsáveis por diferentes aspectos do funcionamento do sistema. Cada módulo possui responsabilidades específicas e interage com os demais componentes de maneira estruturada, garantindo que o fluxo de execução do jogo ocorra de forma consistente.

De forma geral, a arquitetura do sistema é composta pelos seguintes componentes principais:

- **Core (Núcleo da Engine)**
Responsável pela inicialização do sistema, gerenciamento do ciclo de execução e coordenação geral dos módulos da *engine*.
- **Renderer (Sistema de Renderização)**
Responsável pela renderização de elementos gráficos bidimensionais na tela, incluindo sprites e outros componentes visuais utilizados no jogo.
- **Entity System (Sistema de Entidades)**

Responsável pelo gerenciamento das entidades presentes no ambiente do jogo, permitindo a criação, atualização e remoção de objetos durante a execução.

- ***Collision System (Sistema de Colisão)***

Responsável pela detecção de colisões entre entidades, permitindo que interações físicas ou lógicas sejam processadas pelo sistema.

- ***Input System (Sistema de Entrada)***

Responsável por capturar e processar eventos de entrada do usuário provenientes de dispositivos como teclado e mouse.

- ***Resource Manager (Gerenciador de Recursos)***

Responsável pelo carregamento, armazenamento e gerenciamento de recursos utilizados pela aplicação, como imagens e arquivos de mapa.

- ***Scene Manager (Gerenciador de Cenas)***

Responsável pela organização do jogo em diferentes estados ou cenas, permitindo a transição entre diferentes contextos da aplicação.

A interação entre esses componentes permite que o sistema execute o ciclo fundamental de funcionamento de um jogo, que envolve o processamento de entrada do usuário, atualização da lógica do jogo e renderização dos elementos gráficos.

A organização modular da arquitetura facilita a compreensão da estrutura do sistema, além de permitir que novos componentes ou funcionalidades sejam adicionados sem comprometer a organização geral da *engine*.

16 ESTILO ARQUITETURAL

A arquitetura da *engine* foi projetada com base em um modelo modular, no qual o sistema é dividido em componentes independentes responsáveis por funcionalidades específicas. Esse estilo arquitetural permite que cada módulo seja desenvolvido e mantido de forma isolada, reduzindo o acoplamento entre os componentes e facilitando a evolução do sistema ao longo do tempo.

A utilização de uma arquitetura modular contribui para a organização do código-fonte e melhora a manutenibilidade do *software*, uma vez que alterações em um módulo tendem a causar menor impacto nos demais componentes do sistema.

Além da modularidade, a arquitetura também adota princípios de organização em camadas. Nesse modelo, os componentes do sistema são organizados de forma hierárquica, onde determinados módulos fornecem serviços para outros componentes localizados em níveis superiores da estrutura.

De maneira geral, a arquitetura da *engine* pode ser organizada em três níveis principais:

- **Core Layer (Núcleo do Sistema)**

Responsável pela inicialização da *engine*, gerenciamento do ciclo de execução do jogo e coordenação geral dos módulos do sistema.

- **System Layer (Camada de Sistemas)**

Composta pelos módulos responsáveis por funcionalidades específicas da *engine*, como renderização gráfica, gerenciamento de entidades, sistema de colisão, processamento de entrada do usuário e gerenciamento de recursos.

- **Application Layer (Camada de Aplicação)**

Representa o código do jogo desenvolvido pelo usuário da *engine*, que utiliza os serviços fornecidos pelos módulos internos do sistema.

Essa organização permite separar claramente as responsabilidades de cada parte do sistema, facilitando a compreensão da arquitetura e a manutenção do código.

Outro aspecto importante do estilo arquitetural adotado é a utilização de interfaces bem definidas entre os módulos. Essas interfaces permitem que diferentes componentes se comuniquem de forma estruturada, reduzindo dependências diretas e tornando o sistema mais flexível para futuras modificações.

A adoção desses princípios arquiteturais contribui para a criação de uma *engine* organizada, extensível e adequada para fins educacionais e experimentação no desenvolvimento de jogos 2D.

17 COMPONENTES DO SISTEMA

A arquitetura da *engine* é composta por um conjunto de componentes responsáveis por diferentes funcionalidades essenciais para o funcionamento do sistema. Cada componente possui responsabilidades bem definidas e interage com os demais módulos de forma estruturada, contribuindo para a organização e modularidade da *engine*.

A divisão do sistema em componentes permite reduzir o acoplamento entre as partes do *software*, facilitando a manutenção, a expansão e a reutilização de funcionalidades.

A seguir são apresentados os principais componentes que compõem a arquitetura da *engine*.

17.1 Core

O componente *Core* representa o núcleo da *engine* e é responsável por coordenar o funcionamento geral do sistema.

Entre suas principais responsabilidades estão a inicialização da *engine*, o gerenciamento do ciclo de execução do jogo e a coordenação dos demais módulos que compõem o sistema.

O *Core* também é responsável por controlar o *loop* principal da aplicação, garantindo que as etapas de atualização da lógica do jogo e renderização gráfica ocorram de maneira organizada durante a execução.

17.2 *Renderer*

O componente *Renderer* é responsável pelo processamento e exibição dos elementos gráficos do jogo na tela.

Esse módulo gerencia a renderização de *sprites* e outros elementos bidimensionais utilizados na construção do ambiente do jogo. O sistema de renderização recebe informações das entidades presentes na cena e processa sua representação visual durante cada ciclo de execução do jogo.

17.3 *Entity System*

O *Entity System* é responsável pelo gerenciamento das entidades presentes no ambiente do jogo.

Uma entidade representa qualquer objeto existente no mundo do jogo, como personagens, elementos do cenário ou objetos interativos. Esse componente permite a criação, atualização e remoção de entidades durante a execução da aplicação.

O gerenciamento estruturado das entidades facilita a organização da lógica do jogo e permite que diferentes objetos sejam manipulados de forma consistente.

17.4 Collision System

O *Collision System* é responsável pela detecção de colisões entre entidades presentes no ambiente do jogo.

Esse módulo verifica a interseção entre objetos e fornece informações que podem ser utilizadas pela lógica do jogo para determinar interações, como bloqueios de movimento ou eventos específicos.

A implementação de um sistema de colisão é fundamental para permitir a interação entre elementos presentes no ambiente do jogo.

17.5 Input System

O *Input System* é responsável por capturar e processar eventos de entrada provenientes do usuário.

Esse componente permite que a aplicação receba comandos através de dispositivos como teclado e mouse, convertendo esses eventos em ações que podem ser utilizadas pela lógica do jogo.

O processamento adequado das entradas do usuário é essencial para permitir a interação entre o jogador e o ambiente do jogo.

17.6 Resource Manager

O *Resource Manager* é responsável pelo gerenciamento dos recursos utilizados pela aplicação.

Entre esses recursos estão arquivos de imagem, *sprites* e outros elementos necessários para a execução do jogo. Esse módulo centraliza o carregamento e o armazenamento desses recursos, evitando duplicação e facilitando o controle de memória.

17.7 Scene Manager

O *Scene Manager* é responsável pela organização do jogo em diferentes cenas ou estados.

Cada cena representa um contexto específico da aplicação, como *menus*, fases ou telas de configuração. Esse componente permite controlar a transição entre diferentes partes do jogo de forma estruturada.

18 DESCRIÇÃO DOS MÓDULOS

Esta seção apresenta uma descrição mais detalhada dos módulos que compõem a arquitetura da *engine*, destacando suas responsabilidades principais e sua interação com os demais componentes do sistema.

A divisão da *engine* em módulos permite organizar melhor o código-fonte e garantir que cada parte do sistema possua responsabilidades bem definidas, contribuindo para a manutenção e evolução do *software*.

18.1 Módulo Core

O módulo *Core* é responsável por controlar o funcionamento geral da *engine*. Ele atua como o ponto central de coordenação entre os diferentes componentes do sistema.

Entre suas principais responsabilidades estão:

- Inicialização da *engine*;
- Criação e gerenciamento da janela de execução;
- Controle do *loop* principal do jogo;
- Coordenação da atualização dos sistemas internos da *engine*;
- Encerramento seguro da aplicação.

O *Core* interage diretamente com os demais módulos da *engine*, garantindo que o fluxo de execução ocorra de forma organizada durante cada ciclo do jogo.

18.2 Módulo de Renderização

O módulo de Renderização (*Renderer*) é responsável por exibir os elementos gráficos do jogo na tela.

Esse componente recebe informações sobre os objetos presentes na cena e realiza o processo de renderização dos elementos visuais, como *sprites* e outros objetos gráficos.

Entre suas principais responsabilidades estão:

- Renderização de *sprites*;
- Atualização da tela a cada ciclo de execução;
- Gerenciamento do processo de desenho dos elementos do jogo.

Esse módulo trabalha em conjunto com o sistema de entidades e com o gerenciador de cenas para determinar quais elementos devem ser exibidos durante cada quadro da execução.

18.3 Módulo de Entidades

O *Entity System* é responsável pelo gerenciamento das entidades presentes no ambiente do jogo.

Cada entidade representa um objeto dentro do mundo do jogo, podendo possuir diferentes características e comportamentos.

Entre as responsabilidades desse módulo estão:

- Criação de entidades;
- Atualização de entidades durante o *loop* do jogo;
- Remoção de entidades quando necessário;
- Organização das entidades dentro da cena atual.

Esse sistema permite que a lógica do jogo manipule diferentes objetos de forma estruturada.

18.4 Módulo de Colisão

O *Collision System* é responsável por identificar quando ocorre interação entre entidades presentes no ambiente do jogo.

Esse módulo analisa a posição e os limites das entidades para verificar possíveis interseções entre objetos.

Entre suas principais responsabilidades estão:

- Detecção de colisões entre entidades;

- Identificação de interseções entre objetos;
- Fornecimento de informações para a lógica do jogo sobre colisões detectadas.

Essas informações podem ser utilizadas para controlar movimentação, bloqueios ou eventos específicos dentro do jogo.

18.5 Módulo de Entrada

O *Input System* é responsável por capturar eventos de entrada provenientes do usuário.

Esse módulo recebe sinais gerados por dispositivos como teclado e mouse e os converte em comandos que podem ser utilizados pela lógica do jogo.

Entre suas principais responsabilidades estão:

- Captura de eventos de teclado;
- Captura de eventos de *mouse*;
- Disponibilização dessas informações para o restante da *engine*.

Esse componente permite que o jogador interaja com o ambiente do jogo.

18.6 Módulo de Gerenciamento de Recursos

O *Resource Manager* é responsável pelo carregamento e gerenciamento dos recursos utilizados pela aplicação.

Entre esses recursos estão arquivos de imagem, *sprites* e outros elementos necessários para a execução do jogo.

As principais responsabilidades desse módulo incluem:

- Carregamento de recursos gráficos;
- Armazenamento e gerenciamento desses recursos;
- Fornecimento dos recursos para outros módulos da *engine* quando necessário.

Esse gerenciamento centralizado contribui para melhor controle de memória e organização dos arquivos utilizados pelo jogo.

18.7 Módulo de Gerenciamento de Cenas

O *Scene Manager* é responsável pela organização da aplicação em diferentes cenas.

Cada cena representa um estado específico do jogo, como *menus*, níveis ou telas de pausa.

Entre as responsabilidades desse módulo estão:

- Criação e gerenciamento de cenas;
- Controle da cena atualmente ativa;
- Transição entre diferentes cenas do jogo.

Esse sistema permite estruturar a aplicação de forma organizada, separando diferentes partes do jogo em contextos distintos.

19 FLUXO DE EXECUÇÃO DA *ENGINE*

O fluxo de execução da *engine* descreve a sequência de operações realizadas durante a execução de um jogo desenvolvido utilizando o sistema. Esse fluxo representa o ciclo fundamental de funcionamento da aplicação, conhecido como *loop* principal do jogo.

O *loop* principal é responsável por garantir que o jogo seja atualizado continuamente enquanto a aplicação estiver em execução. Durante cada ciclo do *loop*, a *engine* realiza uma série de etapas que permitem processar a interação do usuário, atualizar o estado do jogo e renderizar os elementos gráficos na tela.

De forma geral, o fluxo de execução da *engine* pode ser dividido nas seguintes etapas:

19.1 Inicialização do Sistema

A execução da *engine* inicia com a fase de inicialização do sistema. Nesse momento, o módulo Core é responsável por configurar os componentes principais necessários para o funcionamento da aplicação.

Durante essa etapa são realizadas operações como:

- A inicialização da *engine*;
- A criação da janela de execução;
- A inicialização dos módulos internos do sistema;
- O carregamento inicial de recursos necessários para o jogo.

Após a conclusão da fase de inicialização, a *engine* entra no *loop* principal de execução.

19.2 Processamento de Entrada

Durante cada ciclo do *loop* do jogo, o *Input System* é responsável por capturar e processar eventos provenientes do usuário.

Esses eventos podem incluir:

- O pressionamento de teclas;
- A movimentação do *mouse*;
- Os cliques de *mouse*.

As informações capturadas são disponibilizadas para a lógica do jogo, permitindo que o jogador interaja com o ambiente virtual.

19.3 Atualização da Lógica do Jogo

Após o processamento das entradas do usuário, o sistema realiza a atualização da lógica do jogo.

Durante essa etapa, o *Entity System* atualiza o estado das entidades presentes no ambiente do jogo. Essa atualização pode incluir:

- A movimentação de personagens;
- A atualização de comportamentos;
- A execução de ações específicas definidas pela lógica do jogo.

Essa etapa garante que o estado do jogo evolua ao longo do tempo.

19.4 Detecção de Colisões

Após a atualização das entidades, o *Collision System* realiza a verificação de possíveis colisões entre os objetos presentes no ambiente do jogo.

Caso sejam detectadas interseções entre entidades, essas informações podem ser utilizadas pela lógica do jogo para determinar interações específicas, como bloqueio de movimento, coleta de objetos ou ativação de eventos.

19.5 Renderização da Cena

Após a atualização da lógica do jogo e a verificação de colisões, o módulo *Renderer* realiza o processo de renderização da cena atual.

Durante essa etapa, os elementos gráficos associados às entidades presentes na cena são desenhados na tela, permitindo que o jogador visualize o estado atual do jogo.

Esse processo ocorre repetidamente a cada ciclo do *loop* principal, atualizando continuamente a imagem exibida na tela.

19.6 Encerramento da Aplicação

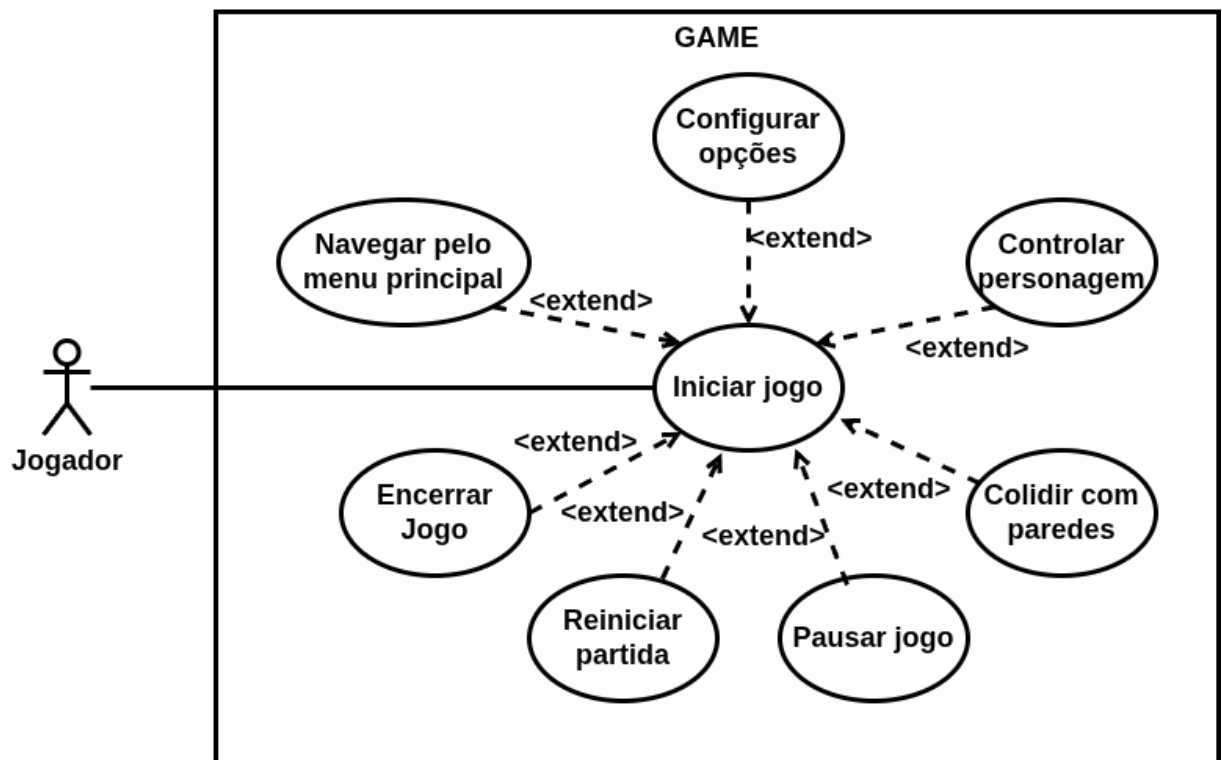
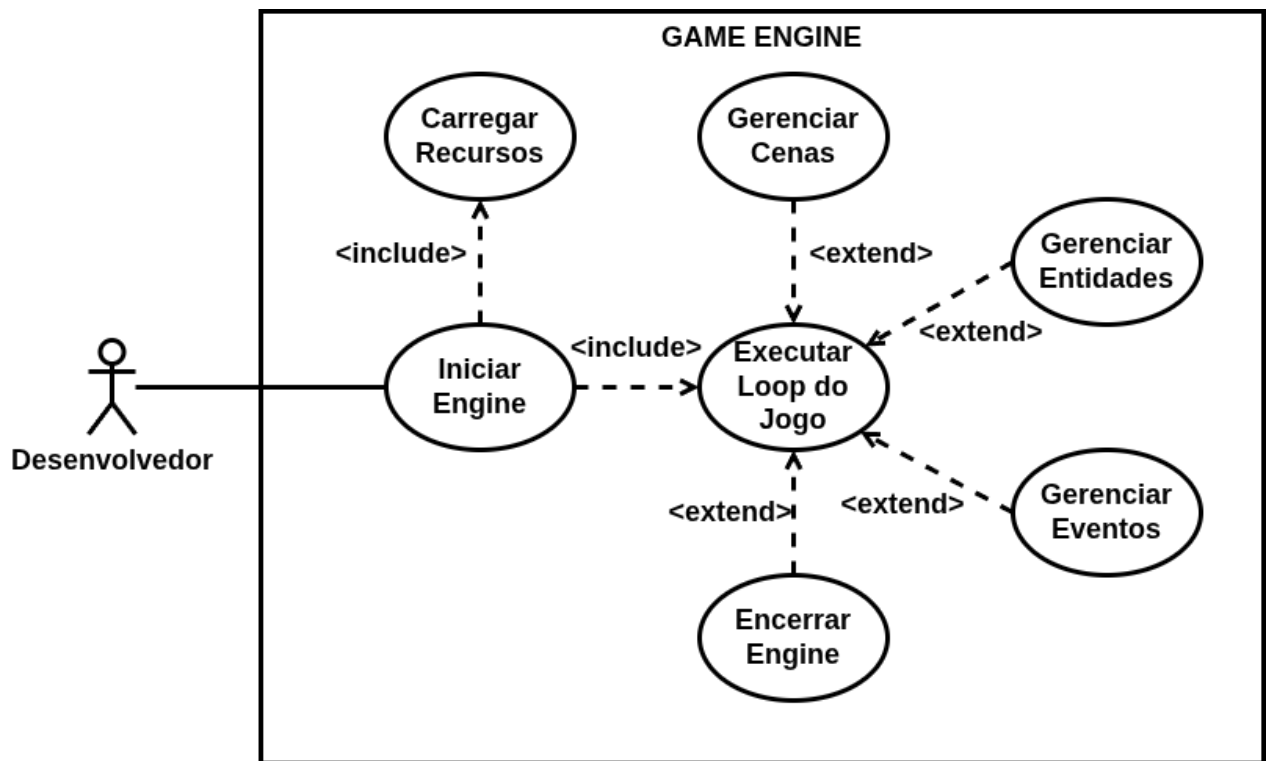
O *loop* principal da *engine* permanece em execução até que seja solicitado o encerramento da aplicação.

Quando isso ocorre, o módulo *Core* é responsável por finalizar a execução do sistema, liberando os recursos utilizados pela *engine* e encerrando a aplicação de forma segura.

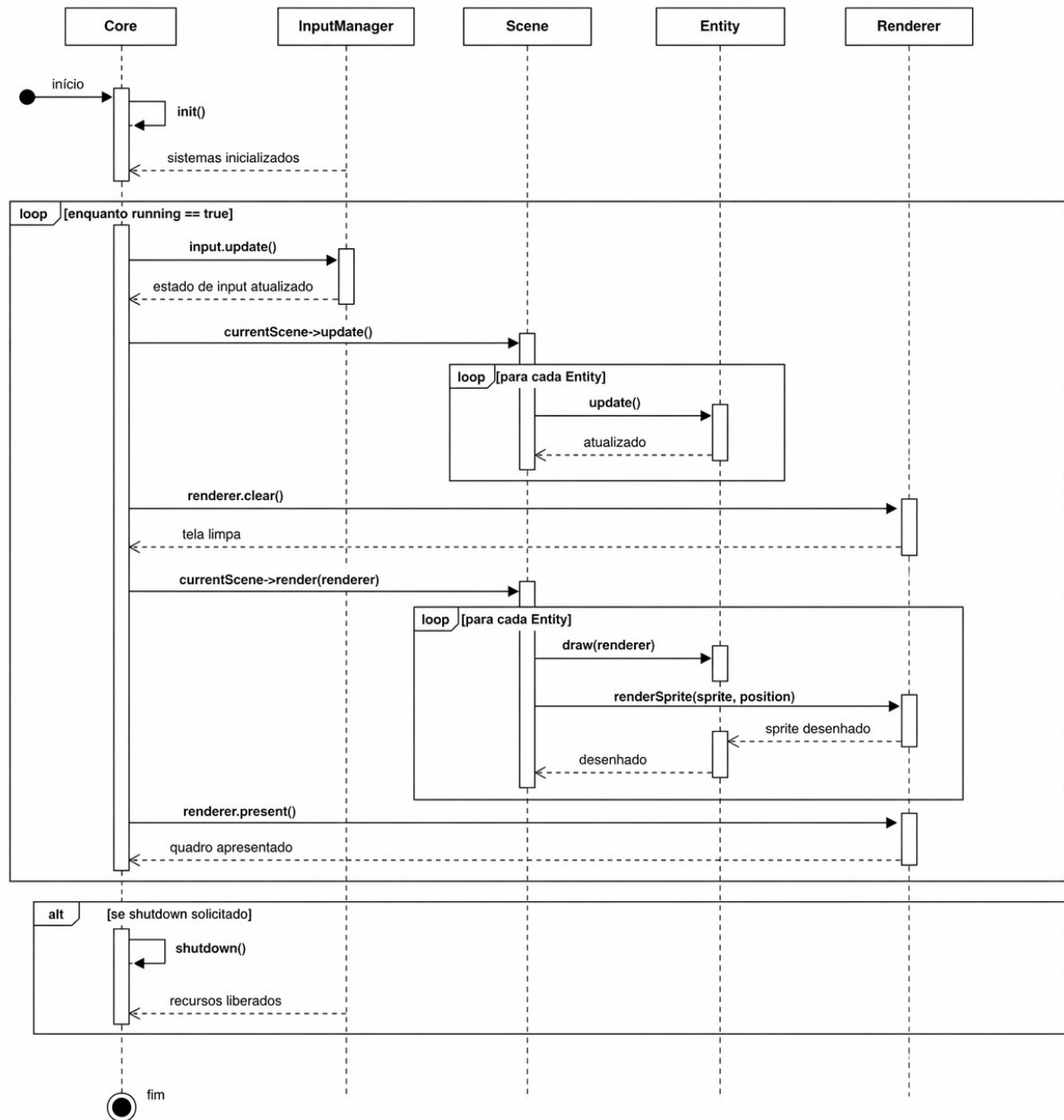
20 DIAGRAMAS

Os diagramas foram estruturado seguindo os princípios de baixo acoplamento e responsabilidade única, utilizando composição para representar dependência forte e associação para relações mais flexíveis, conforme a notação padrão da UML.

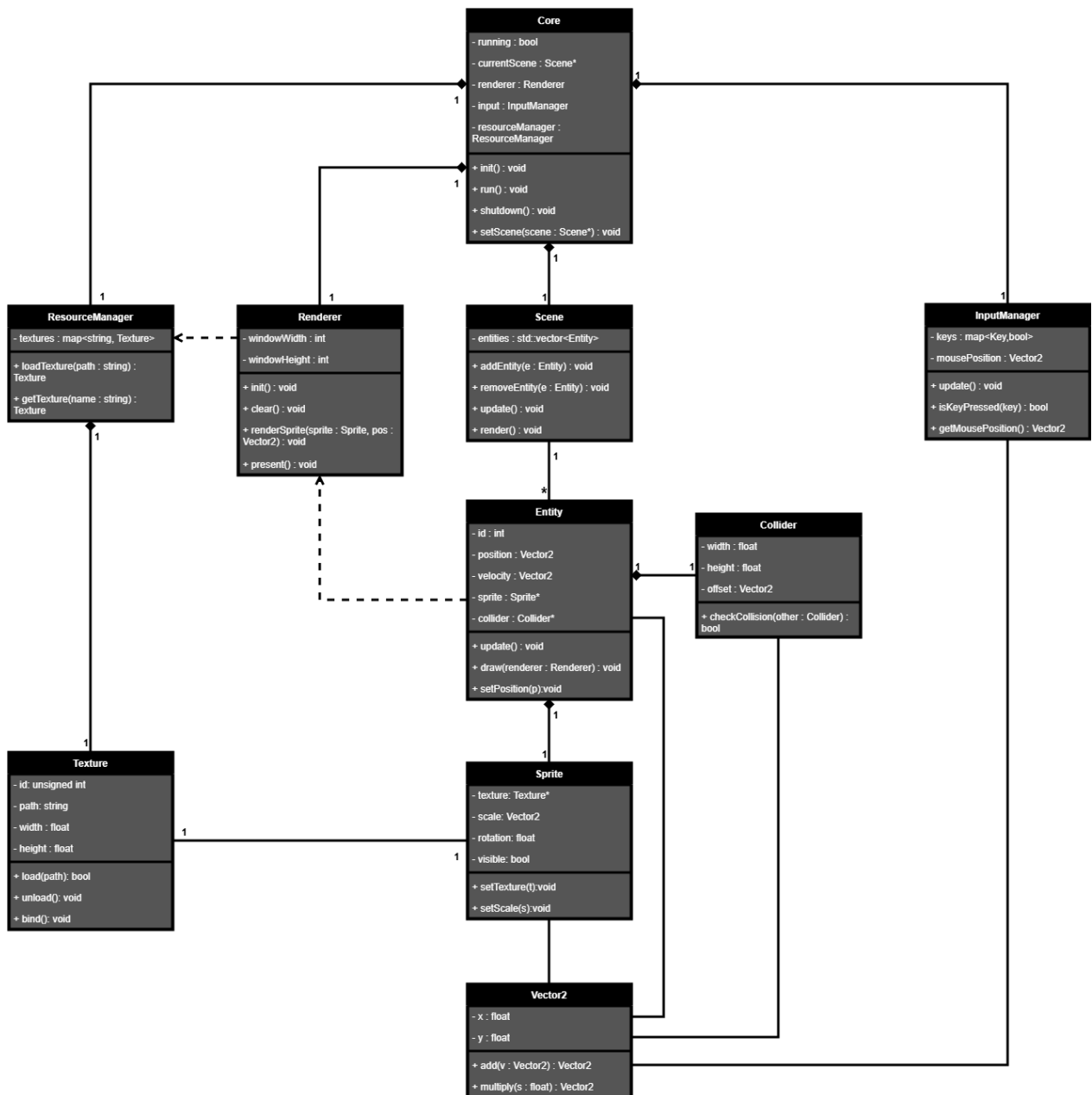
20.1 Caso de Uso



20.2 Diagrama de Sequencia



20.3 Classe e Relacionamento



REFERÊNCIAS

GAMMA, Erich; HELM, Richard; JOHNSON, Ralph; VLISSIDES, John. Padrões de projeto: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000.

GIL, Antonio Carlos. Métodos e técnicas de pesquisa social. 7. ed. São Paulo: Atlas, 2019.

GREGORY, Jason. Game engine architecture. 3. ed. Boca Raton: CRC Press, 2018.

PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de software: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2020.

SOMMERVILLE, Ian. Engenharia de software. 10. ed. São Paulo: Pearson, 2019.

STROUSTRUP, Bjarne. The C++ programming language. 4. ed. Boston: Addison-Wesley, 2013.